A Causality-Preserving, Stereographically Compactified Worldline Representation for Retarded Liénard–Wiechert Electrodynamics

Tim Huff
Independent Researcher (Creve Coeur, MO, USA)
Software: hopf-hough v6.2.0, **[TODO: Zenodo software DOI for v6.2.0]** (MIT License)

Version 6.2 — October 2026

Abstract

We describe and verify `hopf-hough`, an open-source browser tool and library that evaluates the exact retarded Liénard–Wiechert (LW) field of moving point charges at every pixel of an observation plane, in real time on the GPU, and marks the pixels where the result should not be trusted. The mathematics is established; the contribution is a verified single-precision implementation. Each pixel solves, for every source, the light-cone equation $f(R) = R - \|\mathbf{x} - \mathbf{z}(R)\| = 0$ on a compact per-source history and evaluates the LW field from the interpolated history. This gather-side formulation is efficient when sources are few and observation points many. Three ingredients make it accurate in WebGL2 float32: (i) the history is resampled onto shells of retarded radius $R = c(t - t_{\mathrm{ret}})$ placed by the map $R = L\tan(\theta/2)$; (ii) cubic Hermite segments are limited by a closed-form certificate on the Bernstein coefficients of $\|\mathbf{z}'(R)\|^2$, which guarantees $\|\mathbf{z}'\| \le \beta_{\max} < 1$ and hence one retarded root per observer; (iii) the root function is evaluated in double-float arithmetic at the bracketing knot only, and inside the segment by a cancellation-free incremental form that takes the chord deficit $1 - |\bar{\mathbf{v}}|^2$ from float64 values computed on the CPU. Version 6.1 attributed the remaining float32 error of the version-6 kernel to knot-level root-function arithmetic. We show that at $\gamma \gtrsim 70$ it comes instead from the float32 offset inside the segment, and that the new kernel (v6f) removes it while doing 0.20–$0.24\times$ the arithmetic of version 6 on the readback path. Against the exact field (no softening) at the float32-represented observer positions, with $M_T = 512$ shells, the relative L_2 error across a synchrotron pulse is **[TODO: v6f, $\gamma = 22.4$]** at $\gamma = 22.4$ and **[TODO: v6f, $\gamma = 70.7$]** at $\gamma = 70.7$; at $\gamma = 224$, with double-float observer coordinates, it is **[TODO: v6f, 224, 512]** ($M_T = 512$) and **[TODO: v6f, 224, 2048]** ($M_T = 2048$). On exactly sampled preset paths the error at $\gamma = 70.7$ is about 1.4×10^{-4}%. A Python port of the double-precision reference agrees with the independent code PyCharge to $\sim 10^{-11}$ up to $\gamma = 7.1$. In far-field tests no observer left unmarked by the validity indicator is more than 0.68% from the exact field. No new physics is claimed.

Keywords: Liénard–Wiechert fields, retarded time, causality-preserving interpolation, Bernstein polynomials, double-float arithmetic, stereographic compactification, synchrotron radiation, WebGL2

1 Introduction and relationship to prior work

The field of a moving point charge is given in closed form by the Liénard–Wiechert expressions evaluated at the retarded time [1]. Computing it numerically at many observation points is routine in accelerator physics and in teaching. Existing tools take four routes.

1. **Grid solvers (FDTD/PIC)** such as WarpX [2] evolve $(\mathbf{E}, \mathbf{B})$ on a mesh. They handle self-consistent many-particle dynamics, but they suffer numerical dispersion and need spatial resolution of order $R_{\mathrm{orb}}\gamma^{-3}$ to resolve a synchrotron pulse. Radiation diagnostics inside PIC codes, such as RaDiO [3], deposit each particle's LW field forward onto a detector in space and time.

2. **Wavefront methods.** Shintake's wavefront method, with a real-time two-dimensional animation of synchrotron and undulator radiation [4, 5], and LANL's CoSyR [6] construct fields from wavefronts emitted along stored trajectories. These scatter from source to field.

3. **Per-observer retarded-time solves.** PyCharge [7] and the accompanying visualization work [8] compute exact time-domain fields from analytic 3D trajectories by solving the retarded-time equation at each observation point, as batch computations (PyCharge 2 is JIT-compiled with JAX and can run on GPUs). The Liénard–Wiechert solver of Ryne *et al.* [9] locates the retarded time on a stored particle history by bisection over the stored values, refines it with Brent's method, and uses double-double arithmetic where round-off matters. Zhao *et al.* [10] bracket and bisect on a stored history and then solve a quadratic on a linear segment. Nakayama, Oda and Yasuda [11] scan a stored worldline for its crossing of an observer's past light cone and solve a quadratic on the linearly interpolated segment, in an interactive teaching visualization. Frequency-domain codes such as SRW [12] compute the radiation in the near field by integrating over emission time.

4. **Coordinate compactification.** Maps such as $R = L\tan(\theta/2)$ are standard in numerical relativity for mapping unbounded domains to finite grids [13]. Related algebraic maps of the half-line are standard in spectral methods [14].

Gather versus scatter. A grid solver stores the field per cell and lets sources scatter into it. The method here stores a compact history per source and lets each observation point gather: it asks each source which event on its worldline lies on the observer's past light cone. The split between per-observer "retarded-time" and per-source "source-time-dominant" formulations is long established for moving-source noise prediction in aeroacoustics [15, 16]. Bisection on a stored history [9, 10] and per-point retarded-time solves [7] are also established. What this work contributes is an engineering combination: a bounded-cost, single-precision GPU kernel with a guaranteed unique root, a compact radial history, a per-pixel validity indicator, and measured, reproducible accuracy. The gather formulation is efficient when the number of sources is modest and observation points are dense. Grid solvers remain the right tool for self-consistent plasmas with $\sim 10^9$ macroparticles.

Contributions

- A closed-form, branch-light causality certificate for cubic Hermite worldline segments based on the Bernstein coefficients of $\|\mathbf{z}'\|^2$ (Proposition 1), and its degree-8 extension to the quintic segments used for exactly known paths (Section 2.7). The cubic certificate is never stricter than the control-vector limiter of version 5 (Theorem 1). On random segment data the version-5 limiter allows a median of 0.26 of the largest blend that actually keeps $\|\mathbf{z}'\| \le \beta_{\max}$ (the new certificate: 0.46). On smooth arcs it fires needlessly and harms accuracy.

- An account of where single precision loses accuracy (Sections 2.4 and 2.6). For the version-5 algorithm it is one subtraction in the root function and absolute knot storage; double-float arithmetic at the knots removes most of that. The error that remains in version 6 at $\gamma \gtrsim 70$ comes from the float32 offset inside the segment. A cancellation-free incremental root function, with the chord deficit from the CPU in float64, removes it. On Apple's Metal compiler the double-float steps require guarding every rounding step against reassociation, since the usual guard (multiplying by a uniform equal to 1) does not survive.

- An open, single-file browser tool and TypeScript library with a built-in verification suite, an independent cross-check against PyCharge, and a shader-side validity indicator.

2 Formulation

2.1 Retarded Liénard–Wiechert field and a cancellation-free kernel

For a point charge q with trajectory $\mathbf{z}(t_{\text{ret}})$, $\boldsymbol{\beta} = \dot{\mathbf{z}}/c$ and $\dot{\boldsymbol{\beta}} = \ddot{\mathbf{z}}/c$, let $R = c(t - t_{\text{ret}})$ be the retarded radius at observation time t, so that $d\mathbf{z}/dR = -\boldsymbol{\beta}$. The retarded radius R_{ret} for an observer at $\mathbf{x}$ solves

$$f(R) = R - \|\mathbf{x} - \mathbf{z}(R)\| = 0, \qquad \hat{\mathbf{n}} = \frac{\mathbf{x} - \mathbf{z}(R_{\text{ret}})}{\|\mathbf{x} - \mathbf{z}(R_{\text{ret}})\|}, \tag{1}$$

and the field is [1]

$$\mathbf{E}(\mathbf{x}, t) = \frac{q(1-\beta^2)}{\kappa^3 R^2}(\hat{\mathbf{n}} - \boldsymbol{\beta}) + \frac{q}{c\,\kappa^3 R}\hat{\mathbf{n}} \times \left[(\hat{\mathbf{n}} - \boldsymbol{\beta}) \times \dot{\boldsymbol{\beta}}\right], \qquad \kappa = 1 - \hat{\mathbf{n}} \cdot \boldsymbol{\beta}, \qquad \mathbf{B} = \hat{\mathbf{n}} \times \mathbf{E}. \tag{2}$$

Charges move in the x–y plane and the field is evaluated in that plane, where $\mathbf{E}$ is in-plane and $\mathbf{B}$ is along z. The tool offers an optional Plummer softening $R^2 \to R^2 + \epsilon^2$ for display near charges; all accuracy figures in this paper use $\epsilon = 0$. (Version 5 compared against a reference that applied the same softening, so its "exact" reference was not the exact field; see Section 7.)

To avoid cancellation in $1 - \hat{\mathbf{n}} \cdot \boldsymbol{\beta}$ and $1 - \beta^2$ as $\beta \to 1$, the history stores the proper velocity $\mathbf{u} = \gamma\boldsymbol{\beta}$, recovers $1 - \beta^2 = (1 + \|\mathbf{u}\|^2)^{-1}$, and evaluates

$$\kappa = \frac{1-\beta^2}{1+\beta} + \frac{1}{2}\beta\,\|\hat{\mathbf{n}} - \hat{\boldsymbol{\beta}}\|^2, \qquad \hat{\mathbf{n}} - \boldsymbol{\beta} = \frac{1-\beta^2}{1+\beta}\,\hat{\mathbf{n}} + \beta\,(\hat{\mathbf{n}} - \hat{\boldsymbol{\beta}}). \tag{3}$$

The radiation term is evaluated as

$$\hat{\mathbf{n}} \times \left[(\hat{\mathbf{n}} - \boldsymbol{\beta}) \times \dot{\boldsymbol{\beta}}\right] = (\hat{\mathbf{n}} - \boldsymbol{\beta})(\hat{\mathbf{n}} \cdot \dot{\boldsymbol{\beta}}) - \kappa\,\dot{\boldsymbol{\beta}}, \tag{4}$$

which follows from $\hat{\mathbf{n}} \cdot (\hat{\mathbf{n}} - \boldsymbol{\beta}) = \kappa$, with κ and $\hat{\mathbf{n}} - \boldsymbol{\beta}$ from Eq. (3) and $\hat{\mathbf{n}}$ from the separation $\mathbf{x} - \mathbf{z}$ formed in double-float arithmetic (Section 2.4). The cross-product form evaluated in v6.0.0 rebuilds $\kappa \sim 1/(2\gamma^2)$ from $O(1)$ float32 terms. With Eq. (4) and the Newton fix of Section 2.5, the version-6 pulse-sweep error at $M_T = 512$ fell from 0.0095% to 0.0077% at $\gamma = 22.4$ and from 0.069% to 0.036% at $\gamma = 70.7$ (software rendering).

2.2 Stereographic radial shells

Each source's history is resampled onto M_T cell-centred shells $\theta_m = (m + \frac{1}{2})\Delta\theta$, $\Delta\theta = \pi/M_T$, at

$$R(\theta) = L\tan\left(\frac{\theta}{2}\right), \qquad \Delta R(\theta) = \frac{L^2 + R^2}{2L}\,\Delta\theta. \tag{5}$$

Spacing is nearly uniform for $R \lesssim L$, where radiation forms, and grows as R^2 beyond. The outermost shell is at $L\tan(\frac{\pi}{2} - \frac{\Delta\theta}{4}) \approx 4LM_T/\pi$. The map is one of the algebraic maps of the half-line onto a finite interval used in spectral methods; Boyd's rational Chebyshev functions on $[0, \infty)$ use the related map $y = L\cot^2(t/2)$ [14]. It provides reach within a fixed texture, not uniform resolution to infinity: accuracy falls once ΔR no longer resolves the motion (Section 4.3). Shells are sampled from an append-only ring buffer in retarded time, so the textures are a resampled view of that buffer, refreshed each step. In the configuration used here the buffer holds $c\,\Delta t\,N_{\text{ring}} \approx 260$ length units of history. Shells placed beyond that depth carry no valid history and are flagged (Section 2.8), so the usable reach is the smaller of the outermost shell and the buffer depth. For preset motions the shells can instead hold exact samples of the path (Section 2.7).

2.3 Causality certificates for cubic Hermite segments

On a segment $[R_1, R_2]$, $\Delta R = R_2 - R_1$, with knot positions $\mathbf{z}_1, \mathbf{z}_2$ and knot velocities $\boldsymbol{\beta}_1, \boldsymbol{\beta}_2$ ($\|\boldsymbol{\beta}_i\| \le \beta_{\max} < 1$), write $\mathbf{v} = (\mathbf{z}_2 - \mathbf{z}_1)/\Delta R$ and assume the chord is subluminal, $\|\mathbf{v}\| < \beta_{\max}$. Consider a cubic Hermite curve with endpoint R-derivatives blended toward the secant by $\alpha \in [0,1]$,

$$\mathbf{V}_0(\alpha) = (1-\alpha)\mathbf{v} - \alpha\boldsymbol{\beta}_1, \qquad \mathbf{V}_2(\alpha) = (1-\alpha)\mathbf{v} - \alpha\boldsymbol{\beta}_2, \qquad \mathbf{V}_1(\alpha) = \mathbf{v} + \alpha(2\mathbf{v} + \boldsymbol{\beta}_1 + \boldsymbol{\beta}_2). \tag{6}$$

It interpolates the knots for every α, and its derivative is exactly the quadratic Bernstein form $\mathbf{z}'(s) = \sum_{k=0}^{2} B_{k,2}(s)\mathbf{V}_k(\alpha)$, $s = (R - R_1)/\Delta R$.

Theorem 1 (Control-vector certificate, version 5)**.** *$\mathbf{V}_0$ and $\mathbf{V}_2$ lie in the ball $\|\cdot\| \le \beta_{\max}$ for all α by convexity. If α^* is the largest $\alpha \in [0,1]$ with $\|\mathbf{V}_1(\alpha)\| \le \beta_{\max}$ (a closed-form quadratic root), then $\|\mathbf{z}'(R)\| \le \beta_{\max}$ on the segment by the convex-hull property of Bernstein polynomials.*

The bound is a known technique: velocity limits through Bernstein control points are standard in CNC and robot trajectory planning [17, 18], and blending tangents toward the secant is a vector form of monotonicity-preserving Hermite interpolation [19]. Applied to retarded time it is useful but conservative. For a circular arc of turning angle φ, $\|\mathbf{V}_1\| = \beta(1 + \varphi^2/8 + O(\varphi^4))$, so the limiter fires on smooth arcs whose true speed never exceeds β. On the version-5 benchmark it fired on 4–87% of segments (14–87% at $M_T = 128$) that were never superluminal, and it caused most of the $\gamma = 70.7$, $M_T = 128$ error (Section 4).

Proposition 1 (Squared-speed certificate)**.** $\|\mathbf{z}'(s)\|^2 = \sum_{k=0}^{4} B_{k,4}(s)\, c_k$ *with*

$$c_0 = \mathbf{V}_0\cdot\mathbf{V}_0, \quad c_1 = \mathbf{V}_0\cdot\mathbf{V}_1, \quad c_2 = \tfrac{1}{3}\left(2\,\mathbf{V}_1\cdot\mathbf{V}_1 + \mathbf{V}_0\cdot\mathbf{V}_2\right), \quad c_3 = \mathbf{V}_1\cdot\mathbf{V}_2, \quad c_4 = \mathbf{V}_2\cdot\mathbf{V}_2.$$

Each $c_k(\alpha)$ is quadratic in α, and $c_k(0) = \|\mathbf{v}\|^2 < \beta_{\max}^2$. Let α_{sq} be the smallest $\alpha \in (0,1]$ at which some $c_k(\alpha) = \beta_{\max}^2$ (or 1 if none); if $\|\mathbf{v}\| = \beta_{\max}$, set $\alpha_{sq} = 0$. Then for every $\alpha \le \alpha_{sq}$, $\|\mathbf{z}'\| \le \beta_{\max}$ on the segment, and $\alpha_{sq} \ge \alpha^*$.

Proof. The product of two quadratic Bernstein expansions is a quartic Bernstein expansion with the stated coefficients; since $B_{k,4} \ge 0$ and $\sum_k B_{k,4} = 1$, $\|\mathbf{z}'\|^2 \le \max_k c_k \le \beta_{\max}^2$. c_0 and c_4 never exceed $\beta_{\max}^2$ by Theorem 1, and by continuity no c_k exceeds it before its first crossing. If all $\|\mathbf{V}_k\| \le \beta_{\max}$ then by Cauchy–Schwarz every $c_k \le \beta_{\max}^2$, so no c_k exceeds $\beta_{\max}^2$ on $[0, \alpha^*]$; outside degenerate tangential contact this gives $\alpha_{\mathrm{sq}} \ge \alpha^*$. □

Computing α_{sq} costs three quadratic solves. The guarantee holds in exact arithmetic. In a float32 emulation it held for $\beta \le 0.99999$ with $\beta_{\max} = 0.9999995$, but it was violated by up to $\sim 1 - \beta_{\max}$ at $\beta = 0.999999$, so $\beta_{\max} - \beta$ must exceed a few float32 ulps. With the speed bound on every segment, $f'(R) = 1 + \hat{\mathbf{n}}\cdot\mathbf{z}'(R) \ge 1 - \beta_{\max} > 0$, so f is strictly increasing on the stored domain and each observer has exactly one retarded root there. The limited worldline is C^0 everywhere and C^1 at a knot when neither neighbouring segment is limited; where a limiter acts, the velocity direction can be discontinuous at the knot. A globally C^1 subluminal cubic does not always exist (for alternating chords $\pm c$ with free knot slopes it requires $c \le 2\beta/3$), whereas raising the degree restores it. Section 2.7 uses quintic segments where the path is known exactly.

2.4 Single-precision accuracy and its cause

The version-5 algorithm. We attributed the error of the version-5 float32 algorithm, in a CPU emulation, by promoting one stage at a time to float64 [25]. At $\gamma = 70.7$ ($M_T = 512$), rounding only the root-function arithmetic $\mathbf{x} - \mathbf{z}$, $\|\cdot\|$ and $R - \|\cdot\|$ to float32 produces 12.4% error, and storing knot positions in absolute float32 coordinates produces 2.4%. Every other stage (κ and the LW kernel, Newton conditioning, the velocity and acceleration interpolation, shell radii) contributes at most 0.05%. The cause is conditioning: near the pulse crest $f'(R) \approx \kappa \sim \gamma^{-2}$, so an absolute error δ in f moves the retarded point by δ/κ, and float32 positions near $|\mathbf{x}| \sim 1$ carry $\delta \sim 6 \times 10^{-8}$.

Version 6: double-float arithmetic at the knots. Knot positions are stored as pairs $\mathbf{z} = \mathbf{z}_{\mathrm{hi}} + \mathbf{z}_{\mathrm{lo}}$ of float32 values. Chords are formed from both parts, and $\mathbf{x} - \mathbf{z}_1$, its length and $R_1 - \|\mathbf{x} - \mathbf{z}_1\|$ are evaluated with error-free transformations (double-float arithmetic [20, 21]). This lowers the version-5 error about 100-fold. Error-free transformations only work if the compiler evaluates them as written. Apple's Metal shader compiler (reached through ANGLE) does not. On an M1 Max, an earlier version that guarded some intermediates by multiplying by a uniform equal to 1 fell back to float32 accuracy, because unguarded sums such as $(a - x) + (b - y)$ were regrouped as $(a + b) - (x + y)$. Guarding with the uniform on every operation was still not enough. The shader therefore routes every rounding step through an identity the compiler cannot see through (reinterpreting the float as an integer and XOR-ing it with a uniform equal to 0), and splits operands for the exact products by masking mantissa bits. A probe page in the repository tests each building block on the user's GPU against exact results.

What limits version 6 (corrected in v6.2). Version 6.1 stated that the remaining float32 error came from the root function and that everything other than the knot-level root arithmetic could stay in float32. That is not what limits version 6 at high γ. In the version-6 kernel, the offset inside the segment, $\Delta\mathbf{z}(s) = \mathbf{z}(R) - \mathbf{z}_1$, and $\Delta R = R - R_1$ are float32 Hermite evaluations that are added to the double-float separation. $\Delta\mathbf{z}$ therefore carries an absolute error of about $\varepsilon\,\|\Delta\mathbf{z}\|$, where $\varepsilon = 2^{-24}$. The part of f that changes across the segment is

$$\Delta R - \|\Delta\mathbf{z}\| = \Delta R\,\frac{1 - |\bar{\mathbf{v}}|^2}{1 + |\bar{\mathbf{v}}|}, \qquad \bar{\mathbf{v}} = \Delta\mathbf{z}/\Delta R,$$

and the chord deficit $1 - |\bar{\mathbf{v}}|^2$ is of order γ^{-2} near the crest. Its float32 relative error, and hence the relative error of the root increment, is of order $\varepsilon\gamma^2$. Double-float arithmetic at the knots does not remove this term. The evidence is direct. A diagnostic build of the new kernel (Section 2.6, before its direction fix) that forms only the chord deficit naively in float32, as $1 - |\Delta\mathbf{z}/\Delta R|^2$, reproduces the version-6 error in every case tested (for example 1.05% at $\gamma = 224$, $M_T = 512$, on the known path) [25]. The version-6 rows of Table 2 at $\gamma = 70.7$ and 224 therefore measure this floor, not interpolation of the history.

Two further float32 effects above $\gamma \approx 100$ were corrected in version 6. (i) The chord-speed clamp that guards against superluminal input compared float32 differences of adjacent knots. At $M_T = 2048$ their relative rounding ($\sim 3\times10^{-5}$) exceeds $1-\beta$, so physical knots were wrongly moved. The clamp now uses the full high/low positions in double precision. (ii) The Newton derivative was floored at 5×10^{-5}, larger than the true f' at $\gamma = 224$. The floor is now $\frac{1}{2}(1 - \beta_{\mathrm{max}})$, which the certificate guarantees (in exact arithmetic) is below f'.

How much accuracy is available. At $\gamma = 70.7$ the pulse crossed by the benchmark sweep is about 15 float32 ulps wide in observer position, and moving an observer by one ulp changes $\mathbf{E}$ by up to 71% of its rms value. Rounding the observer coordinates alone shifts the exact field by 3.2% (Table 2). A float32-addressed method can therefore only be judged against the exact field at the rounded point, which is the metric used here, or by placing observers with double-float coordinates. The library's readback API supports double-float observers (the rendered view uses float32 pixel coordinates), and they are required above $\gamma \approx 100$.

2.5 Root search (version-6 kernel)

Because f is monotone at the stored knots (subluminal chords), the solver first bisects over knot indices, evaluating f at knots only (three texel fetches each, no spline evaluation). It then runs safeguarded Newton inside the bracketing segment on the certified cubic, iterating to convergence (at most 12 steps). The segment index and the fraction within it are kept as separate values, so float32 resolution of a combined coordinate never limits the retarded point. Since v6.0.0 the safeguard keeps a closed bracket $[s_{\mathrm{lo}}, s_{\mathrm{hi}}]$, accepts a Newton step that lands on either end, falls back to

bisection otherwise, and stops when $f = 0$ exactly. In v6.0.0 a step that reached the root exactly could be bisected away from it. The same fix is in the JAX port.

Display path. For on-screen rendering only, the kernel builds the segment coefficients and the certificate once per pixel and source and reuses them across Newton steps; this halves the texture fetches with bit-identical results. It also bisects over knots in float32 with one double-float check of the bracket, and evaluates the root function in double-float without a double-float square root, as $f = (R^2 - |\mathbf{d}|^2)/(R + |\mathbf{d}|)$, with the numerator in double-float and the division in float32. An instrumented count gives 2398 operations per pixel and source against 7788 for the full path, and the rendered output differs from the full solve by at most 3/255 per channel. Readback (the suite, the probe page, exports and every table here) always takes the full path.

2.6 The v6f kernel: float32 incremental root function

The v6f kernel keeps double-float arithmetic only where it is needed: $f_1 = R_1 - \|\mathbf{d}_1\|$ and $\mathbf{d}_1 = \mathbf{x} - \mathbf{z}_1$ at the bracketing knot. Inside the segment it uses a float32 form with no cancellation. It is compiled from the version-6 shader source with one preprocessor switch, and the v6 and v5 programs are bit-identical to before.

Incremental root function. Let knot 1 be the bracketing knot. Because R increases into the past, knot 1 is the later event. Write $\Delta\mathbf{z} = \mathbf{z}(R) - \mathbf{z}_1$, $\ell = \|\Delta\mathbf{z}\|$, $\rho = R - R_1 = s\,\Delta R$, $L_1 = \|\mathbf{d}_1\|$ and $L = \|\mathbf{d}_1 - \Delta\mathbf{z}\| = \|\mathbf{x} - \mathbf{z}(R)\|$. Then

$$f(R) = f_1 + \underbrace{[\rho - \ell]}_{\geq 0} + \underbrace{[\ell + L_1 - L]}_{\geq 0}, \tag{7}$$

where the first bracket is non-negative because the segment is subluminal and the second by the triangle inequality. Both are formed without cancellation:

$$\rho - \ell = \frac{\rho\, D}{1 + \ell/\rho}, \qquad D = 1 - |\bar{\mathbf{v}}|^2, \quad \bar{\mathbf{v}} = \Delta\mathbf{z}/\rho, \tag{8}$$

$$\ell + L_1 - L = \begin{cases} \dfrac{2\,|\mathbf{d}_1 \times \Delta\mathbf{z}|^2}{(\ell L_1 - \mathbf{d}_1\cdot\Delta\mathbf{z})\,(\ell + L_1 + L)}, & \mathbf{d}_1\cdot\Delta\mathbf{z} \leq 0, \\ \dfrac{2\,(\ell L_1 + \mathbf{d}_1\cdot\Delta\mathbf{z})}{\ell + L_1 + L}, & \mathbf{d}_1\cdot\Delta\mathbf{z} > 0. \end{cases} \tag{9}$$

The second line of Eq. (9) follows from $(\ell + L_1)^2 - L^2 = 2(\ell L_1 + \mathbf{d}_1 \cdot \Delta\mathbf{z})$. The first then follows from $(\ell L_1)^2 - (\mathbf{d}_1 \cdot \Delta\mathbf{z})^2 = |\mathbf{d}_1 \times \Delta\mathbf{z}|^2$, which in two dimensions is the scalar cross product. The case split depends on the time direction. At the pulse crest the charge moves toward the observer, so the earlier point $\mathbf{z}(R)$ is farther away. $\Delta\mathbf{z}$ is then nearly antiparallel to $\mathbf{d}_1$, $\mathbf{d}_1 \cdot \Delta\mathbf{z} \approx -\ell L_1$, and $\ell L_1 + \mathbf{d}_1 \cdot \Delta\mathbf{z}$ cancels. The well-conditioned denominator is $\ell L_1 - \mathbf{d}_1 \cdot \Delta\mathbf{z} \approx 2\ell L_1$. A form written for a parameter running forward in time, with the roles of the two ends exchanged, has its cancelling sum in the denominator in this convention; that was corrected in testing. Each term of Eqs. (7)–(9) is a float32 sum of non-negative quantities, so reassociation cannot cancel it and these steps carry no compiler barrier. A 50-digit check over 2000 random triangles agreed with the direct form to 10^{-47} [25].

Chord deficit. D must not be formed as $1 - |\bar{\mathbf{v}}|^2$ from a float32 $\bar{\mathbf{v}}$, which is exactly the version-6 floor. $\bar{\mathbf{v}}(s)$ is the mean of the hodograph over $[0, s]$, $\bar{\mathbf{v}} = \sum_k I_k(s)\mathbf{V}_k$ with $I_k(s) = s^{-1}\int_0^s B_{k,2}$. Substituting Eq. (6) writes it as an affine combination of three points, the chord velocity $\mathbf{P}_v = \mathbf{v}$ and the knot points $\mathbf{P}_1 = -\boldsymbol{\beta}_1$, $\mathbf{P}_2 = -\boldsymbol{\beta}_2$:

$$\bar{\mathbf{v}}(s) = \lambda_v\mathbf{P}_v + \lambda_1\mathbf{P}_1 + \lambda_2\mathbf{P}_2, \qquad \lambda_1 = \alpha(1-s)^2, \quad \lambda_2 = -\alpha s(1-s), \quad \lambda_v = 1 - \lambda_1 - \lambda_2, \tag{10}$$

exactly, for any blend α. (In the Bernstein indexing of Eq. (6), $\lambda_1 = \alpha(I_0 - I_1)$ and $\lambda_2 = \alpha(I_2 - I_1)$, with $I_0 - I_1 = (1-s)^2$ and $I_2 - I_1 = -s(1-s)$.) For weights with $\sum_i \lambda_i = 1$, of any sign,

$$1 - \Big|\sum_i \lambda_i \mathbf{P}_i\Big|^2 = \sum_i \lambda_i D_i + \sum_{i<j} \lambda_i \lambda_j \, |\mathbf{P}_i - \mathbf{P}_j|^2, \qquad D_i = 1 - |\mathbf{P}_i|^2. \tag{11}$$

The kernel takes $D_v = 1 - |\mathbf{v}|^2$ per knot from the CPU, computed in float64 from the high/low knot positions with the same chord clamp as the shader. It is stored in a texture channel that was unused. The knot deficits are $D_{1,2} = (1 + |\mathbf{u}_{1,2}|^2)^{-1}$ from the stored proper velocity. The pair terms are squared lengths of small differences, $|\mathbf{v} + \boldsymbol{\beta}_1|^2$, $|\mathbf{v} + \boldsymbol{\beta}_2|^2$ and $|\boldsymbol{\beta}_1 - \boldsymbol{\beta}_2|^2$, formed once per segment. No term subtracts two quantities close to 1. For the quintic segments of Section 2.7 the hodograph is quartic with control points $\mathbf{Q}_0, \ldots, \mathbf{Q}_4$. Because $\sum_k \mathbf{Q}_k = 5\mathbf{v}$, $\mathbf{Q}_2$ can be eliminated, giving an affine combination of $(\mathbf{v}, \mathbf{Q}_0, \mathbf{Q}_1, \mathbf{Q}_3, \mathbf{Q}_4)$ with $\lambda_k = \alpha_q(I_k - I_2)$, $I_k = s^{-1}\int_0^s B_{k,4}$. Here $\mathbf{Q}_0 = -\boldsymbol{\beta}_1$, $\mathbf{Q}_4 = -\boldsymbol{\beta}_2$, $\mathbf{Q}_1 = \mathbf{Q}_0 + \mathbf{w}_1$ and $\mathbf{Q}_3 = \mathbf{Q}_4 - \mathbf{w}_2$, with $\mathbf{w}_k = \frac{1}{4}\Delta R\, \mathbf{a}_k$ and $\mathbf{a}_k = \mathbf{z}''(R_k)$ the stored acceleration. So $D_{Q_1} = D_1 + 2\boldsymbol{\beta}_1 \cdot \mathbf{w}_1 - |\mathbf{w}_1|^2$ and $D_{Q_3} = D_2 - 2\boldsymbol{\beta}_2 \cdot \mathbf{w}_2 - |\mathbf{w}_2|^2$. Segments limited by the certificate ($\alpha_q < 1$) are covered by the same form.

Knot bisection in float32 with a double-float bracket check. The knot bisection uses the float32 function $g_m = R_m - \|\mathbf{x}_{\mathrm{hi}} - \mathbf{z}_{\mathrm{hi},m}\|$ (two texel fetches). It differs from the true $f_m = f(R_m)$ by at most a rounding bound δ, a few ulps of the coordinates plus the neglected low parts. The knot values are strictly increasing, $f_{m+1} - f_m \geq \Delta R_m(1 - \|\mathbf{v}_m\|) > 0$. A sign of g can therefore be wrong only at knots with $|f_m| \leq \delta$, and by monotonicity these form one contiguous run next to the root. Every bisection decision at a knot outside that run is correct, so the float32 bracket ends inside or next to that run. The kernel then evaluates f in double-float at both bracket knots and moves the bracket outward one knot at a time (at most 16 steps) until $f_{\mathrm{lo}} < 0 \leq f_{\mathrm{hi}}$ holds in double-float. Because f is monotone along the knots, this is the bracket a double-float bisection finds. The float32 pass only changes how many double-float evaluations are needed. In 18 test cases the output was identical to double-float bisection (maximum difference 0), with 2.0–2.7 double-float and 11–13 float32 knot evaluations per pixel and source, against 11–13 double-float knot evaluations for version 6 [25].

Direction rounding and the remaining field floor. With the chord deficit fixed, v6f on exactly sampled paths was limited at about $0.6 \times 3\varepsilon\gamma$ (for example 7×10^{-6} relative at $\gamma = 70.7$). Promoting stages one at a time to float64 in an emulation that matches the GPU locates this floor in the float32 rounding of directions, not in the κ formula ($\leq 2 \times 10^{-7}$) or the $1/R$ factors [25]. Near the crest $\hat{\mathbf{n}}$, $\hat{\boldsymbol{\beta}}$ and the chord are parallel to within about $1/\gamma$. A rotation by ε of any one of them is then a relative error of about $\varepsilon\gamma$ in $1 - \hat{\mathbf{n}} \cdot \hat{\boldsymbol{\beta}}$, in $\hat{\mathbf{n}} \cdot \mathbf{a}$ and in the root's $\mathbf{d}_1 \times \Delta\mathbf{z}$. Five or six stages contribute comparable amounts, so no single promotion helps by more than about 20%. The fix takes every cancelling cross or dot product against the double-float separation $\mathbf{d}_1 = \mathbf{d}_{\mathrm{hi}} + \mathbf{d}_{\mathrm{lo}}$, once per pixel and source, with exact (two-product) multiplications. The segment is linear in its data $(\mathbf{v}, \boldsymbol{\beta}_1, \boldsymbol{\beta}_2, \mathbf{a}_1, \mathbf{a}_2)$, so the scalars $\psi_k = \mathbf{d}_1 \times \mathbf{V}_k$ (hodograph control points) and $\chi_k = \mathbf{d}_1 \times \mathbf{C}_k$ (position control points relative to $\mathbf{z}_1$) give

$$\mathbf{d}_1 \times \mathbf{z}'(s) = \sum_k B_k(s)\, \psi_k, \qquad \mathbf{d}_1 \times \Delta\mathbf{z}(s) = \sum_k B_k(s)\, \chi_k, \tag{12}$$

in the Bernstein (or, for the cubic position, Hermite) basis of the segment, with no rounding of a direction. The second sum is used in Eq. (9). At the root, with $\hat{\boldsymbol{\beta}} = -\mathbf{z}'/|\mathbf{z}'|$, $\hat{\boldsymbol{\beta}}_\perp$ the in-plane perpendicular and ϕ the angle from $\hat{\boldsymbol{\beta}}$ to $\hat{\mathbf{n}}$,

$$\sin\phi = \frac{\mathbf{d}_1 \times \mathbf{z}' - \Delta\mathbf{z} \times \mathbf{z}'}{|\mathbf{z}'|\, L}, \quad 1 - \cos\phi = \frac{\sin^2\phi}{1 + \cos\phi}, \quad \hat{\mathbf{n}} - \hat{\boldsymbol{\beta}} = \sin\phi\, \hat{\boldsymbol{\beta}}_\perp - (1 - \cos\phi)\, \hat{\boldsymbol{\beta}}, \tag{13}$$

and then $\kappa = (1 - \beta) + \beta(1 - \cos\phi)$ and $\hat{\mathbf{n}} - \boldsymbol{\beta} = (1 - \beta)\hat{\mathbf{n}} + \beta(\hat{\mathbf{n}} - \hat{\boldsymbol{\beta}})$, as in Eq. (3). Likewise $\hat{\mathbf{n}} \cdot \mathbf{a}(s) = [\sum_k w_k(s)(\mathbf{d}_1 \cdot \mathbf{a}_k) - \Delta\mathbf{z} \cdot \mathbf{a}(s)]/L$, with w_k the interpolation weights and $\mathbf{d}_1 \cdot \mathbf{a}_k$ exact

products. The fix runs only when $|\mathbf{u}| > 4$ at a knot of the segment ($\gamma > 4.1$), since below that the floor is under $\sim 3 \times 10^{-7}$. On the known path it lowers the error 2–17×, to 1.2×10^{-6} at $\gamma = 70.7$ and 4–6×10^{-6} at $\gamma = 224$ for $M_T \geq 512$. That is close to the floor set by float32 storage of **u** and **a**. Ring-buffer histories are limited by interpolation and do not change (Table 1).

Cost. One double-float root evaluation costs about 519 float32 operations, and the incremental root costs 57 (cubic) or 87 (quintic). On the readback path v6f does 0.20–$0.24\times$ the arithmetic of version 6 (hand count times measured evaluation counts). On the display path, where version 6 already bisects in float32, the estimate is 0.76–$0.81\times$. In SwiftShader, with 32 sources, v6f took 0.45–$0.53\times$ the frame time of version 6 on both paths. The direction fix adds 10–15% there even when no source is above the gate, because SwiftShader evaluates the gated branch; a compile-time switch removes it. SwiftShader timings are relative only. On an Apple M1 Max with 32 sources in the default scene, version 6 renders at 62 fps and version 5 at 120 fps (display-capped); v6f renders at **[TODO: M1 Max fps, v6f, 32 sources, default scene]**.

		known path (exact knots)		live ring buffer	
γ	M_T	v6	v6f	v6	v6f
70.7	128	0.12%	8.5×10^{-5}%	—	—
70.7	512	0.027%	1.4×10^{-4}%	0.043%	0.024%
70.7	2048	0.0078%	1.2×10^{-4}%	0.012%	0.0015%
224	128	4.4%	0.0035%	—	—
224	512	1.05%	5.5×10^{-4}%	0.68%	0.068%
224	2048	0.28%	4.0×10^{-4}%	0.31%	0.0052%

Table 1: Relative L_2 error on the 256-point pulse sweep (one synchrotron source, no softening, double-float observers), for the version-6 and v6f kernels in the app, software rendering (SwiftShader). "Known path": shells hold exact samples joined by quintic segments (Section 2.7). "Live": the ring-buffer history after 600 live steps. The v6 known-path errors equal those of a diagnostic build of v6f (before the direction fix) that forms only the chord deficit naively in float32. The live rows are limited by interpolation of the history once the floor is removed. Source: [25], **[TODO: path of these scripts and logs in the v6.2.0 release]**.

2.7 Histories: dragged sources and known paths

Physical dragging. In the app, a dragged charge does not follow the pointer directly. The pointer sets an intent, and the charge follows it on a worldline whose position, velocity and acceleration are consistent. A two-loop controller limits the closing velocity by braking distance and the acceleration to a budget of $1.25\sqrt{M_T/128}$ per time unit. It bounds the jerk and keeps the speed below a cap less than c. The stored acceleration equals the applied change of velocity, and positions follow the trapezoid rule. On release the charge returns to its preset path through the same dynamics, and control passes back only when the two worldlines coincide, so the history has no jump.

Known-path mode. When a source follows a preset whose position, velocity, acceleration and jerk are known in closed form, its shells can hold exact samples of the path instead of resamplings of the ring buffer. A segment between two exact knots is a quintic Hermite curve matching position, velocity and acceleration at both knots. Its hodograph is quartic with control points $\mathbf{Q}_0, \ldots, \mathbf{Q}_4$, so $\|\mathbf{z}'\|^2$ has degree-8 Bernstein coefficients $c_k = \sum_{i+j=k} \binom{4}{i}\binom{4}{j}\, \mathbf{Q}_i \cdot \mathbf{Q}_j \big/ \binom{8}{k}$. With $\mathbf{Q}_k(\alpha) = \mathbf{v} + \alpha(\mathbf{Q}_k - \mathbf{v})$ each c_k is quadratic in α, and the certificate of Proposition 1 carries over: take the first α at which some c_k reaches β^2_{max}. Where the certificate does not limit a segment the worldline is C^2. A limited segment no longer matches the endpoint derivatives and is C^0 there, as the cubic is.

Knots whose retarded time falls in a dragged stretch come from the ring buffer, and so do purely live sources. On the orbit preset ($\beta = 0.8$, 4 sources, $M_T = 512$) the known-path and ring-buffer fields differ by at most 3.3×10^{-6} RMS through a drag, the return and the hand-off, so the hand-off adds no visible step. An *Auto* shell count picks M_T from the view's largest retarded radius: 128 for the app's default view, or 512 above $\gamma = 100$. With the version-6 kernel, known-path mode reached the pulse-sweep accuracy of the ring buffer at $M_T = 2048$ with 2–16× fewer shells at $\gamma \le 22$, 64× fewer over the default view at $\gamma \le 71$, and 4× fewer at $6 < R < 20$. At $\gamma \ge 71$ the version-6 sweep was limited by the chord-deficit floor, which v6f removes (Table 1).

2.8 Validity indicator

For each source the shader computes, at the retarded point,

$$q = \max\left(\frac{\Delta R\, \gamma\, |\dot{\beta}_\perp|}{c\, \max(\beta^2, 0.04)},\ \frac{6\, \|\Delta \mathbf{u}\|}{\bar{u} + 0.1},\ q_{\text{acc}} \right).$$

The first term is shell spacing times γ over the local radius of curvature $c\beta^2/|\dot{\beta}_\perp|$, with β^2 floored at 0.04 ($\beta = 0.2$); without the floor it flagged charges at rest. The second is the relative change of proper velocity across the segment. The third, added for dragged histories, is

$$q_{\text{acc}} = 7.5\, \frac{|\hat{\mathbf{n}} - \boldsymbol{\beta}| + \kappa}{\kappa^3 R}\, \frac{\max\left(\|\Delta^2 \mathbf{a}\|,\ \|(\boldsymbol{\beta}_2 - \boldsymbol{\beta}_1)/\Delta R + \tfrac{1}{2}(\mathbf{a}_1 + \mathbf{a}_2)\| \right)}{\|\mathbf{E}_{\text{source}}\|}.$$

It converts two acceleration errors into an estimated radiation-field error relative to the source's own field. The first is a jump in the stored acceleration, seen as the largest second difference $\Delta^2\mathbf{a}$ over the four interpolation knots, which the cubic smooths across the jump's light sphere. The second is stored acceleration inconsistent with the change of stored velocity across the segment ($d\boldsymbol{\beta}/dR = -\mathbf{a}$ with $\mathbf{a} = \mathbf{z}''$). An estimated error of 0.2 maps to $q = 1.5$. The indicator reports $q = -1$ when the retarded point lies beyond the outermost shell or the segment reaches past the ring-buffer depth. It raises q inside the first shell and on shells damped by the optional Nyquist taper. The app hatches pixels with $q > 1.5$ or $q < 0$. The thresholds are heuristic; Section 4.3 measures how well they predict the error. The indicator changes only the mask, never the field values.

3 Verification protocol

Tables 2–4, the straight-line and targeted limiter tests and the cost figures come from `tests/gpu/tables.mjs`. The raw data are in `results/tables.json`, and the version-6 and v6f values of Table 2 are in `results/table2_v6_2.tsv` **[TODO: confirm file and script names in v6.2.0]**. The error attribution, the limiter statistics, the far-field estimates and the preset defect come from the scripts and saved outputs in `validation/analysis/`, and the PyCharge comparison from `validation/pycharge/`.

1. **Reference.** The exact field is computed in float64 directly on the analytic trajectory (32-step bracketed bisection on t_{ret}, then up to 12 safeguarded Newton steps) with $\epsilon = 0$. With softening off, a Python port of this reference agrees with PyCharge 2.0.1 [7], which obtains velocity and acceleration by automatic differentiation of the position. The largest relative difference is 1.3×10^{-11}, for circular orbits ($\beta = 0.5, 0.9, 0.99$, including pulse sweeps), harmonic oscillation and straight-line motion. The TypeScript reference reproduces the version-5 float64 values exactly. The reference is not externally checked above $\gamma = 7.1$.

2. **Benchmark.** A charge on a circular orbit of radius 0.85 ($c = 2.35$). A ring buffer ($\Delta t = 0.00675$) is seeded and advanced 16 steps, and the shells are populated from it. Observers: 256 points at fixed distance from the orbit centre crossing the pulse emitted tangentially at retarded radius 1.15 (half-width $\max(10^{-6}, 4\gamma^{-3})$ rad), and a 64-point ring at radius 1.15.

3. **Readback.** Fields are read back from an `RGBA32F` framebuffer (`gl.readPixels`). The tables use WebGL2 with the SwiftShader software renderer, which implements IEEE float32. With the v6.0.0 kernel, the in-app suite on an Apple M1 Max (Metal) reproduced the software-rendered pulse-sweep column to within run-to-run scatter (e.g. 0.0089% and 0.059% at $\gamma = 22.4$ and 70.7, $M_T = 512$; `results/suite_m1max.tsv`). For the v6.2.0 kernels: **[TODO: M1 Max suite results for v6 and v6f, file name]**. Other GPUs should be checked with the suite.

4. **Regression.** The single-file solver deposited with version 5 is kept in the repository. The TypeScript port reproduces all 154 values of its verification suite exactly in the same browser engine. The version-5 column in Table 2 is that deposited shader run on the same history, against the unsoftened reference. The v5 and v6 programs of v6.2.0 give bit-identical readback and rendered frames to the versions before v6f was added, on 10 scenes.

4 Results

4.1 Accuracy across Lorentz factors

β	γ	M_T	**v6**	**v6, df64 obs.**	**v6f**	**v6f, df64 obs.**	version-5 algorithm	ring (v6f)	observer rounding alone
0.8	1.67	128	[T2]	[T2]	[T2]	[T2]	0.0067%	[T2]	2.2×10^{-5}%
0.8	1.67	512	[T2]	[T2]	[T2]	[T2]	0.001%	[T2]	
0.8	1.67	2048	[T2]	[T2]	[T2]	[T2]	0.0034%	[T2]	
0.95	3.20	128	[T2]	[T2]	[T2]	[T2]	0.017%	[T2]	3.1×10^{-4}%
0.95	3.20	512	[T2]	[T2]	[T2]	[T2]	0.0019%	[T2]	
0.95	3.20	2048	[T2]	[T2]	[T2]	[T2]	0.0075%	[T2]	
0.99	7.09	128	[T2]	[T2]	[T2]	[T2]	0.039%	[T2]	0.0034%
0.99	7.09	512	[T2]	[T2]	[T2]	[T2]	0.0097%	[T2]	
0.99	7.09	2048	[T2]	[T2]	[T2]	[T2]	0.015%	[T2]	
0.999	22.4	128	[T2]	[T2]	[T2]	[T2]	0.26%	[T2]	0.11%
0.999	22.4	512	[T2]	[T2]	[T2]	[T2]	0.22%	[T2]	
0.999	22.4	2048	[T2]	[T2]	[T2]	[T2]	0.29%	[T2]	
0.9999	70.7	128	[T2]	[T2]	[T2]	[T2]	48%	[T2]	3.2%
0.9999	70.7	512	[T2]	[T2]	[T2]	[T2]	7.6%	[T2]	
0.9999	70.7	2048	[T2]	[T2]	[T2]	[T2]	7.8%	[T2]	
0.99999	224	128	—	[T2]	—	[T2]	—	[T2]	> 100%
0.99999	224	512	—	[T2]	—	[T2]	—	[T2]	
0.99999	224	2048	—	[T2]	—	[T2]	—	[T2]	

Table 2: Relative L_2 error of GPU readback against the exact LW field (no softening), ring-buffer history. Sweep: 256 observers crossing the synchrotron pulse; ring: 64 observers at radius 1.15. Columns without "df64" round observers to float32 and compare with the exact field at the rounded point; "df64 obs." places them with double-float coordinates. At $\gamma = 224$ float32 observer coordinates cannot resolve the pulse (last column), so only the df64 columns are meaningful. "v6" is the version-6 kernel of v6.2.0 (with the radiation term of Eq. (4) and the Newton fix), which differs from the v6.0.0 kernel reported in version 6.1 of this paper. The version-5 algorithm uses a float32 root function, the Theorem 1 limiter and bisection on the spline; it was not designed for $\gamma > 100$. **[TODO: fill every [T2] cell from results/table2_v6_2.tsv; bold the headline cells]**

[TODO: Rewrite this paragraph from the new numbers.] Relative to the version-5 solver, the error of v6f falls **[TODO: ratio]**$\times$ at $\gamma = 70.7$ and **[TODO: ratio]**$\times$ at $\gamma = 22.4$ for $M_T \geq 512$. The largest change is at $M_T = 128$, $\gamma = 70.7$ (48% to **[TODO: v6f]**), where the version-5 limiter was active across the pulse. At $\gamma \leq 7$ all kernels are accurate, and the remaining error is interpolation of the history, which converges with M_T. At $\gamma = 70.7$ and 224 the difference between the v6 and v6f columns is the chord-deficit floor of Section 2.4. On the ring-buffer history the v6f error there is limited by interpolation of the history (Table 1).

128 shells, $R \in [0, 220]$	sweep	ring	spacing at $R = 1.15$	first spacing
Stereographic $L\tan(\theta/2)$, $L = 1.35$	**0.038%**	0.012%	0.029	0.017
Logarithmic $r_0(e^{\alpha u} - 1)$	0.33%	0.080%	0.081	0.0035
Quadratic $R_{\max} u^2$	1.6%	0.69%	0.27	0.027
Uniform	$> 100\%$	$> 100\%$	1.72	1.72

Table 3: Same memory and reach, $\beta = 0.99$, prefilter off for every grid (version 5 applied it only to the stereographic grid, and its quadratic-grid entry of 0.21% does not reproduce). The stereographic map's advantage here follows from Eq. (5): near-uniform spacing across the region $R \lesssim L$ that contains the orbit and the observation radius, where the tested logarithmic grid concentrates shells near $R = 0$. Other parameter choices for the competing grids may narrow the gap. Version-6 kernel. **[TODO: confirm unchanged in the v6.2.0 suite]**

4.2 Radial grid at equal memory

4.3 Far field and the validity indicator

M_T (outermost shell)	$R = 1.15$	$R = 3$	$R = 10$	$R = 30$	$R = 100$	$R = 200$
128 (220)	0.0063% (0)	0.61% (1)	2.9% (64)	$> 100\%$ (64)	$> 100\%$ (64)	$> 100\%$ (64)
512 (880)	0.0015% (0)	0.011% (0)	0.20% (29)	31% (64)	$> 100\%$ (64)	$> 100\%$ (64)
2048 (3520)	6.4×10^{-5}% (0)	0.0019% (0)	0.010% (0)	0.18% (64)	$> 100\%$ (64)	$> 100\%$ (64)

Table 4: Circular orbit, $\beta = 0.9$: relative L_2 error on rings of radius R around the orbit centre, and in parentheses the number of the 64 observers marked by the validity indicator. The first column gives the outermost shell; the ring buffer holds 260 length units of history. Across all far-field and straight-line cases run ($\beta = 0.5$ and 0.9; $M_T = 128$, 512, 2048; taper on and off), the L_2 error over unmarked observers on a ring is at most 0.37%, and the worst single unmarked observer is 0.68%. Version-6 kernel with the indicator of Section 2.8 (`docs/ACCURACY.md`). **[TODO: confirm with the v6.2.0 suite and v6f]**

Accuracy falls with distance once the shell spacing, which grows as R^2, no longer resolves the orbital motion; the stored history reaches far beyond the radius at which it stays accurate. The indicator is conservative: it marks some observers whose error is below 1%. For unaccelerated motion its curvature and acceleration terms vanish, so the ring-buffer test is what protects it. With $\beta = 0.5$ straight-line motion and observers 50–800 units away, every observer with a retarded point beyond the buffer is marked (errors 2–69%), and every unmarked one is accurate to 3×10^{-5}%. An earlier draft lacked this test and left such errors unmarked. The optional Nyquist taper of version 5 damps the radiation term on coarse shells. It is off by default, and the shells it modifies are marked.

Dragged histories. Before the acceleration terms were added, the indicator marked 0–35% of the observers with error above 1% on dragged histories. The test drags a source along a pointer path with jumps, holds, rapid reversals and a fast circle, under the drag rules of Section 2.7, with 96×96 observers over $[-3, 3]^2$ and the exact field computed from the recorded history. The indicator now marks 99%, 96% and 99% of them at $M_T = 128$, 512 and 2048. The 99th-percentile error of unmarked observers is below 1% at every M_T (0.9%, 0.4%, 0.05%). On smooth presets the new terms add at most 0.5% marked observers, and with the bend floor, accurate observers near a charge at rest are no longer marked (35% to 11% marked at $M_T = 128$).

4.4 When the causality limiter matters

On smooth histories resampled onto the shells, the unlimited cubic rarely exceeds c. In the synchrotron benchmark its peak speed equals the knot speed. For hyperbolic motion (constant proper acceleration over 100 time units, $\gamma \approx 11$ at the observation time) it reaches $\|\mathbf{z}'\| = 1.057$, but only on the outermost shell ($R \in [73, 220]$, $M_T = 128$). In two such runs (proper acceleration 0.25 and 0.15), 256 observers placed so that their retarded points fall in that segment see 73% and 22% error with the squared-speed certificate, 96% and 49% with Theorem 1, and 93% and 68% with no limiter; the indicator marks all of them. On under-resolved segments a limiter can also raise the error: in the version-5 float64 analysis the squared-speed certificate gave 296% against 70% without a limiter ($\gamma = 22.4$, $M_T = 128$, $R = 8$). The certificate's role is therefore a guarantee, one root per observer so the search cannot take a wrong branch, at no cost on resolved histories. It does not make an under-resolved history accurate.

4.5 Cost

In software rendering (640 $\times$ 400 pixels, 16 sources, $M_T = 512$, three repetitions) the v6.0.0 production kernel took 1.1–1.8$\times$ the time of the version-5 algorithm, with large run-to-run variation; an operation count estimates about 2.7$\times$ more float32 arithmetic per root-function evaluation for the double-float version. Software timings say little about GPU cost. On an Apple M1 Max GPU, with the v6.0.0 kernel, a 256×256 grid takes 5–7 ms for one source and 20 ms for 16 sources at $M_T = 2048$, including upload and readback; each additional source costs about 1 ms per 65,536 observers (`validation/benchmarks/results_webgl_m1max.json`). The relative cost of v6f is given in Section 2.6; **[TODO: M1 Max timings for v6f, if measured]**. Cost is proportional to the number of observers times the number of sources, as for any direct sum of LW fields. There is no hierarchical (multipole-type) acceleration, so the method suits few sources and many observation points.

4.6 Comparison with PyCharge

To separate the algorithm from the language, we ported the method to JAX (float64) and ran it against PyCharge 2.0.1 [7] on the same CPU, the same problems (charges on circular orbits of radius 0.85) and the same 256×256 grid, against the exact field (`validation/benchmarks/`). PyCharge was run both with its default solver and with the cheapest fixed-point/Newton budgets that converge on the full grid.

case (65,536 points)	PyCharge, default	PyCharge, cheapest converged	this method (JAX), $M_T = 2048$	its error
$\beta = 0.9$, 1 source	0.27 s	0.040 s	0.038 s	1.1×10^{-6}
$\beta = 0.99$, 1 source	0.28 s	0.066 s	0.061 s	6.2×10^{-6}
$\beta = 0.999$, 1 source	0.53 s	0.18 s	0.034 s	2.4×10^{-5}
$\beta = 0.9999$, 1 source	0.33 s	0.34 s	0.039 s	2.9×10^{-5}
$\beta = 0.9$, 16 sources	5.1 s	0.61 s	0.66 s	1.0×10^{-6}

Table 5: Float64 on a 2-core CPU, medians of three runs after compilation. PyCharge is exact when it converges (10^{-11}–10^{-14}); the method here interpolates a sampled history. Errors from the v6.0.0 JAX port; with the Newton fix most fall slightly and the $\beta = 0.999$ entry rises 1.4–1.55$\times$ (`validation/benchmarks/README.md`).

The two are tied for $\gamma \lesssim 7$ and for 16 sources. At $\gamma = 22$ and 71 the method is 5$\times$ and 9$\times$ faster, because PyCharge's fixed-point iteration needs more steps as $\beta \to 1$, while the knot search and Newton refinement here have a cost independent of γ. The trade is accuracy: a few times 10^{-5} instead of machine precision. The second difference is precision. PyCharge requires float64, which consumer GPUs run at a small fraction of float32 speed and Apple GPUs do not support. Forced to float32, PyCharge's error on the pulse sweep is 0.027%, 0.86% and 27% at $\gamma = 7.1$, 22.4 and 70.7. For the float32 WebGL kernels ($M_T = 512$) it is 0.0023%, 0.0095% and 0.069% (v6.0.0), and **[TODO: v6f, $\gamma = 7.1$, 22.4, 70.7]** (v6f). PyCharge also handles 3D and self-consistent dynamics, which

this method does not. This method works from any sampled history rather than a differentiable analytic trajectory.

5 Limitations and scope

1. **Two dimensions and prescribed motion.** Charges move in a plane and do not respond to each other's fields; there is no radiation reaction. Nothing in the method is specific to two dimensions: the retarded-time solve is a one-dimensional search along each worldline in any dimension, the certificate (Proposition 1) uses only dot products, and the LW expression is already the three-dimensional one. Motion in 3D with arbitrary observation points is therefore a direct extension, but it is not implemented or tested here. In the v6f kernel, Eq. (9) uses the two-dimensional identity $(\ell L_1)^2 - (\mathbf{d}_1 \cdot \Delta\mathbf{z})^2 = |\mathbf{d}_1 \times \Delta\mathbf{z}|^2$, which in 3D holds with the vector cross product.

2. **History, not far-field storage.** The shell textures are a resampled view of an append-only ring buffer in retarded time, refreshed every step. Coarsening old history removes the radiation pulse, which needs full-resolution acceleration. A hand-off to a stored far field on null slices $(\tau, \hat{\mathbf{n}})$ would only be valid beyond $R \gtrsim 1.75\,\gamma\rho/\varepsilon$ for relative error ε (ρ the radius of curvature; $175\,\gamma\rho$ at 1%), and only after removing the Fresnel arrival delay, which requires each source's emission point. Without that correction the 1% radius is $\approx 210\,\rho\gamma^3$, and storage would cost more memory than the worldline, because the far field undoes the γ^3 compression of the emission time. The worldline is close to the most compact representation. (Version 5 proposed such a hand-off at $R \approx R_{\text{Nyq}}$ and a Hopf-bundle representation of the polarization; both are withdrawn. A transverse field over the sphere needs no bundle when stored as a Cartesian vector orthogonal to $\hat{\mathbf{n}}$, and this pattern is the established Cauchy-characteristic extraction [24].)

3. **Very high γ.** The method is tested to $\gamma = 224$. Above $\gamma \approx 100$ observers must be placed with double-float coordinates, which is available through the readback API but not in the rendered view. The remaining v6f floor on exactly sampled paths is set by float32 storage of $\mathbf{u}$ and $\mathbf{a}$; on ring-buffer histories the error is set by interpolation of the history.

4. **Hardware.** Accuracy was measured with an IEEE float32 software renderer and on one GPU (Apple M1 Max). The v6f kernel's float32 sums of non-negative terms carry no compiler barrier. Reassociation should not hurt them, but this is an argument, **[TODO: or: confirmed by the M1 Max suite]**. Other compilers may transform floating-point code in ways the barrier does not anticipate; the probe page and the in-app suite check this on the user's GPU.

5. **Sharing work between pixels.** Each pixel still bisects over the knots of every source. Because the light spheres of successive knots are nested, a per-source segment-index map could in principle be rasterized, as distance cones are rasterized with the depth test to compute Voronoi diagrams [22], or shared over screen tiles, as lights are culled per tile in Forward+ rendering [23]. This is not part of v6.2.0.

6. **Sources faster than the wave speed.** In media or acoustics, sources can outrun the wave and observers see several retarded roots. The certificate does not apply there; counting all roots per segment is a natural extension.

6 Changes from version 6.1

- **Error attribution (Section 2.4).** Version 6.1 attributed the remaining float32 error to knot-level root-function arithmetic. At $\gamma \gtrsim 70$ it comes from the float32 offset inside the segment, through the chord deficit $1 - |\bar{\mathbf{v}}|^2$. The version-6 rows of Table 2 at $\gamma = 70.7$ and 224 measure that floor.

- **New v6f kernel (Section 2.6).** It adds a float64 chord deficit per knot from the CPU, a cancellation-free incremental root function inside the segment, float32 knot bisection with a double-float bracket check, and exact products against the double-float separation for the directions near the crest.
- **Changes to the version-6 kernel since v6.0.0.** These are the radiation term in the form of Eq. (4), the closed-bracket Newton safeguard that holds at $f = 0$ (Section 2.5), and the cheaper display path, which leaves readback unchanged.
- **Validity indicator.** It gains acceleration-jump and consistency terms for dragged histories and a floor on the bend term near rest (Section 2.8). Table 4 is updated accordingly.
- **App.** It adds physical dragging on consistent worldlines and a known-path mode with exact knots, C^2 quintic segments with a degree-8 certificate (C^0 where limited), an automatic shell count and a hand-off to the ring buffer (Section 2.7).
- **Table 2** is remeasured for the v6.2.0 kernels, and Table 1 is new.
- **Prior work.** Citations are added to the retarded-time solvers on stored histories of Ryne *et al.*, Zhao *et al.* and Nakayama *et al.*, to Shintake's real-time radiation animation, RaDiO and SRW, to Boyd's rational maps of the half-line, and to the rasterization techniques of Hoff *et al.* and Forward+. The CoSyR reference is updated to the journal version.

7 Changes from version 5

- The "exact" reference in version 5 applied the Plummer softening ($\epsilon = 0.06$). All errors here are against the unsoftened field.
- Version 5 attributed the single-precision floor to κ^{-3} evaluation and Newton conditioning. The measured cause for that algorithm is the root-function subtraction and absolute knot storage (Section 2.4). The 6–9% GPU-versus-CPU discrepancy reported there is of the same size as the change produced by legal reorderings of float32 operations (5.7–7.3%), so it need not be a hardware effect.
- The Theorem 1 limiter caused most of the $\gamma = 70.7$, $M_T = 128$ error; it is replaced by Proposition 1. Version 5's Float64 convergence column ran without any limiter, so it measured a different algorithm from the GPU path.
- Version 5's far-field result (1.62% at $R_{\text{obs}} = 45$) used a preset that moved at $0.44c$ with no acceleration and jumped back every 4.4 time units. It tested a Coulomb-like field, not radiation, and is replaced by Table 4.
- Some version-5 table entries do not reproduce with its own code: the quadratic grid (4.4% measured vs 0.21% printed) and the $\beta = 0.999$, $M_T = 128$ cell (0.25% vs 2.47%). The max $|\Delta\mathbf{z}|$ column does not match the deposited suite's emission-point error (about 10^{-7}), and two far-field cells (quadratic and logarithmic grids) were not computed by the deposited suite.
- The ensemble ("swarm") preset double-counted the frame rotation in its acceleration (4–15% RMS error in $\dot{\boldsymbol{\beta}}$, depending on the source) and rescaled velocity without position; both are fixed.
- Prior work on aeroacoustic retarded-time formulations, velocity-bounded Bézier curves and PyCharge is now cited. The far-field hand-off and the Hopf-bundle proposal are withdrawn (Section 5).

8 Code and data availability

Every number in this paper not marked as from v6.0.0 was produced by **hopf-hough v6.2.0** (git tag `v6.2.0`, commit **[TODO: commit hash]**), archived at **[TODO: Zenodo software DOI for v6.2.0]**. That release contains the tool, library, tests and the scripts that produce every number here, under the MIT License, together with the version-5 solver, the PyCharge cross-check and the raw results. The app runs from a single HTML file with no installation. Numbers marked v6.0.0 come from **hopf-hough v6.0.0** (tag `v6.0.0`, commit `d87f823`), archived at doi:10.5281/zenodo.23601496, which version 6.1 of this paper cites. Development continues at `https://github.com/timsdsdio/hopf-hough`. Later versions of the code and of the hosted app may behave differently, and results should be reproduced from the tagged release.

Acknowledgments and AI assistance

The ideas in this work are the author's: replacing a global grid with per-source compactified histories, inspired by parameter-space (Hough) transforms, and the direction of the project. The author also designed and steered the multi-agent workflow in which AI agents did the implementation work. AI systems contributed substantially, and all results were produced by code in the repository and checked by the author:

- **Gemini Argon (Google)**: the original WebGL2 shader, the Theorem 1 construction, the cancellation-free kernel, the version-5 verification suite and drafts of earlier versions.
- **Gemini (Google), as an outside reviewer**: the incremental form of the in-segment root function (Section 2.6) began from its suggestion; the working form, with the chord deficit from the CPU and the denominator corrected for the shader's time direction, was found and corrected in testing.
- **Astra (OpenAI)**: adversarial review of earlier drafts and code (six rounds), including the pulse-crossing sweep and the matched-reach grid benchmark.
- **Claude (Anthropic), Claude Fable 5.1 and Claude Opus 5.5**: review of version 5; a multi-agent analysis that identified the limiter's conservatism and the squared-speed certificate, the cause of the version-5 single-precision floor and the double-float fix, the far-field and prior-art issues, and the preset defects; the TypeScript port, production shader, tests, PyCharge cross-check and documentation; a correctness audit of the validity indicator; the JAX port and the head-to-head benchmark against PyCharge; the diagnosis, with a GPU probe page, of the Metal compiler undoing the double-float arithmetic, and the barrier that fixes it. For version 6.2: the radiation-term and Newton fixes, the indicator's acceleration terms, physical dragging, known-path mode and the cheaper display path; the attribution of the version-6 floor to the chord deficit, the v6f kernel and the field-floor fix; and drafting and editing of this version.

References

[1] J. D. Jackson, *Classical Electrodynamics*, 3rd ed. (Wiley, 1999), Ch. 14.

[2] J.-L. Vay *et al.*, "Warp-X: A new exascale computing platform for beam–plasma simulations," *Nucl. Instrum. Methods Phys. Res. A* **909**, 476–479 (2018).

[3] M. Pardal, A. Sainte-Marie, A. Reboul-Salze, R. A. Fonseca and J. Vieira, "RaDiO: an efficient spatiotemporal radiation diagnostic for particle-in-cell codes," *Comput. Phys. Commun.* **285**, 108634 (2023), doi:10.1016/j.cpc.2022.108634; arXiv:2302.14812.

[4] T. Shintake, "New real-time simulation technique for synchrotron and undulator radiations," in *Proc. LINAC 2002*, Gyeongju, Korea, pp. 661–663 (2002).

[5] T. Shintake, "New mathematical method for radiation field of moving charge," in *Proc. EPAC 2002*, Paris, paper WEPRI038, pp. 1667–1669 (2002).

[6] C.-K. Huang, F. Li, H. N. Rakotoarivelo, B. Shen, J. Domine, B. E. Carlsten, G. A. Dilts, R. V. Garimella, T. J. T. Kwan and R. W. Robey, "CoSyR: A novel beam dynamics code for the modeling of synchrotron radiation effects," *Nucl. Instrum. Methods Phys. Res. A* **1034**, 166808 (2022), doi:10.1016/j.nima.2022.166808; arXiv:2109.14901; code at https://github.com/lanl/cosyr.

[7] M. Filipovich and S. Hughes, "PyCharge: An open-source Python package for self-consistent electrodynamics simulations of Lorentz oscillators and moving point charges," *Comput. Phys. Commun.* **274**, 108291 (2022).

[8] M. Filipovich and S. Hughes, "Space-time computation and visualization of the electromagnetic fields and potentials generated by moving point charges," *Am. J. Phys.* **89**, 482–489 (2021).

[9] R. D. Ryne, C. E. Mitchell, J. Qiang, B. Carlsten and N. Yampolsky, "Using a Liénard–Wiechert solver to study coherent synchrotron radiation effects," in *Proc. FEL2013*, New York, NY, USA, paper MOOCNO04, pp. 17–23 (2013).

[10] Y. Zhao, Y. Liang, Y. Zhang, X. Ma and H. Liu, "An electromagnetic particle-particle method for relativistic electron bunch dynamics from early expansion to long-range transport," arXiv:2605.05543 (2026).

[11] D. Nakayama, K. Oda and K. Yasuda, "Seeing through the light cone: Visualizing electromagnetic fields in special relativity," arXiv:2505.20596 (2025).

[12] O. Chubar and P. Elleaume, "Accurate and efficient computation of synchrotron radiation in the near field region," in *Proc. EPAC'98*, Stockholm, paper THP01G, pp. 1177–1179 (1998).

[13] A. Zenginoğlu, "Hyperboloidal foliations and scri-fixing," *Class. Quantum Grav.* **25**, 145002 (2008).

[14] J. P. Boyd, "Orthogonal rational functions on a semi-infinite interval," *J. Comput. Phys.* **70**, 63–88 (1987), doi:10.1016/0021-9991(87)90002-7.

[15] K. S. Brentner, "Numerical algorithms for acoustic integrals — the devil is in the details," AIAA Paper 96-1706 (1996).

[16] K. S. Brentner, "An efficient and robust method for predicting helicopter high-speed impulsive noise," *J. Sound Vib.* **203**, 87–100 (1997).

[17] R. T. Farouki, *Pythagorean-Hodograph Curves: Algebra and Geometry Inseparable* (Springer, 2008).

[18] J. Tordesillas and J. P. How, "MINVO basis: Finding simplexes with minimum volume enclosing polynomial curves," arXiv:2010.10726 (2020).

[19] F. N. Fritsch and R. E. Carlson, "Monotone piecewise cubic interpolation," *SIAM J. Numer. Anal.* **17**, 238–246 (1980).

[20] A. Thall, "Extended-precision floating-point numbers for GPU computation," ACM SIGGRAPH Research Posters (2006).

[21] G. Da Graça and D. Defour, "Implementation of float-float operators on graphics hardware," arXiv:cs/0603115 (2006).

[22] K. E. Hoff III, T. Culver, J. Keyser, M. Lin and D. Manocha, “Fast computation of generalized Voronoi diagrams using graphics hardware,” in *Proc. SIGGRAPH 99* (ACM, 1999), pp. 277–286.

[23] T. Harada, J. McKee and J. C. Yang, “Forward+: Bringing deferred lighting to the next level,” in *Eurographics 2012 — Short Papers*, C. Andujar and E. Puppo (eds.) (The Eurographics Association, 2012), pp. 5–8, doi:10.2312/conf/EG2012/short/005-008.

[24] N. T. Bishop, R. Gómez, L. Lehner, M. Maharaj and J. Winicour, “High-powered gravitational news,” *Phys. Rev. D* **56**, 6298 (1997).

[25] Analysis scripts, saved outputs and reports for the error attribution, the limiter study, the far-field estimates and the v6f kernel: `validation/analysis/` in the software repository **[TODO: add the v6f attribution, field-floor and bracket-check scripts (F, F2, F3) to the release, or cite their location]**.